

2 Oct 2023

Decrementing Function: a way we can prove that an algorithm (specifically, a loop or a recursive fun) terminates.

$$D: \overset{\text{domain}}{\mathbb{S}} \longrightarrow \overset{\text{co-domain}}{\mathbb{N}}$$

We use the natural #s, but you can use ANY well-ordered set as the codomain

Well-ordered set: A set $\mathcal{V}$ ^{with an order} such that ~~any~~ any subset has a minimum element.

- minimum - vs - infimum
↑ must be contained in the set ↑ can go outside the set
the interval from 0 to 1.

$$\inf (0, 1) = 0$$

$$\min (0, 1) \text{ DNE}$$

$\therefore (0, 1), \mathbb{R}$ are not W.O.S.

$\mathbb{N}$ are a W.O.S.

$\mathbb{Q}$ are not a W.O.S.

$\mathbb{Z}$

- $2\mathbb{Z}$ aka, the even #s, are $\subset \mathbb{Z}$
but $\min \{2\mathbb{Z}\}$ DNE.

$\mathcal{S}$ = our state space
current conditions (variables - explicit or implicit)
of running the algorithm
like debugging mode.

A decrementing fn is a fn ~~from~~

$$D: \mathcal{S} \rightarrow \mathbb{N}$$

such that

- At the top of a nested recursion will be strictly smaller than the previous recursive call.
- At the top of a loop, must be strictly smaller than the last time we were at the top of the loop.

Why does this work?

Let X be the ~~the~~ range of D .

So, $X \subset \mathbb{N}$. Since $\mathbb{N}$ is a W.O.S.,

X has a minimum value.

$\therefore$ I can't loop again nor recurse again
once I reach that value.

Example 1: Binary Search

Search ($A \leftarrow$ a sorted array, $q \leftarrow$ query pt)

- 1: if $|A| = 0$
- 2: return FALSE
- 3: else if $|A| = 1$
- 4: return $q = A[1]$
- 5: end if
- 6: mid = $\lfloor \frac{|A|}{2} \rfloor$
- 7: if $q = A[\text{mid}]$
- 8: return TRUE
- 9: else if $q < A[\text{mid}]$
- 10: return search ($A[1 \dots \text{mid}-1], q$)
- 11: else
- 12: return search ($A[\text{mid}+1], q$)
- 13: end else

our decrementing function is given by

$D: S \rightarrow \mathbb{N}$

is defined by

$D(S) = |A|$

} give the whole function

as a sanity check, make sure this is in $\mathbb{N}$

+ check it actually decrements!

Justification: ~~Let $n := |A|$~~ going into the fun.

Then, at recursive calls in 10 + 11, we send a recursive call of at most $\lfloor \frac{n}{2} \rfloor$ and $n \geq 2$.

Try some examples!

- ① Towers of Hanoi
- ② Merge Sort
- ③ Bubble Sort

- a) write or find pseudocode
- b) come up w/ decrementing fun
- c) justify why it decrements / works

Why we need to justify:

$\lceil \frac{n}{2} \rceil$ need not decrement!

e.g., $n=1 \Rightarrow \lceil \frac{n}{2} \rceil = \lceil \frac{1}{2} \rceil = 1$.

This is how infinite loops can happen.

- Looking for
- ① Is the value $\in \mathbb{Z}$?
 - ② Is it ≥ 0 ?
 - ③ Check decrementing

Two cases:

- ① something is obvs. getting smaller.
Use the size of that as your output!
- ② Nothing gets smaller. Here, ^{or} loop iterations.

for $i=1 \dots$ 

 print i

end for

Why does this loop end?

$$D: S \rightarrow \mathbb{N}$$

defined by

$$D(S) = n - i$$

max # times I see the top of a loop

the # of times I've been at the top of a loop.

(4)

```
01 component(G: a graph; s: node in G)
02   Unmark all nodes of G.
03   RV = emptylist
04   Q = emptylist
05   Mark node s and add it to Q
06   while (Q is not empty)
07     p = first(Q)
08     Q = rest(Q)
09     add p to RV
10     for every node n that is a neighbor of p in G
11       if n is not marked
12         mark n
13         add n to the tail of Q
14   return RV
```

Figure 15.2: Return all nodes connected to s